

Principles of OOP and Indian contribution to object-oriented programming:**1)OOP paradigm/Basic concepts of OOP:**

Object-Oriented Programming (OOP) is a programming paradigm focused on objects, which are instances of classes.

1.Classes and Objects: Classes define attributes and methods; objects are their instances.

2.Encapsulation: Bundles data and methods, restricting direct access to protect object integrity.

3.Inheritance: Allows new classes to inherit from existing ones, promoting code reuse.

4.Polymorphism: Enables methods to behave differently based on the object type.

5.Abstraction: Hides complex details, exposing only necessary components.

6.Advantages: Promotes modularity, reusability, scalability, and maintainability in code.

7.Common Languages: Includes Java, C++, Python, Ruby, and C#.

OOP simplifies software development and enhances code organization.

2)OOP languages:

1.Java: A widely-used language that follows OOP principles strictly, known for its portability and strong community support.

2.C++: An extension of C that incorporates OOP features, allowing for both procedural and object-oriented programming.

3.Python: A versatile language that supports multiple programming paradigms, including OOP, with an easy-to-read syntax.

4.C#: Developed by Microsoft, this language is primarily used for developing Windows applications and follows OOP principles closely.

5.Ruby: Known for its elegant syntax, Ruby is designed for simplicity and productivity, emphasizing OOP.

6.Swift: Developed by Apple for iOS and macOS applications, Swift incorporates OOP principles alongside other programming paradigms.

7.Objective-C: An older language used primarily for macOS and iOS development that blends C with OOP features.

8.JavaScript: While originally a prototype-based language, JavaScript has evolved to support OOP concepts, especially with ES6 classes.

9.PHP: A server-side scripting language that supports OOP and is widely used for web development.

10.Dart: A language developed by Google, primarily for building mobile, desktop, and web applications, which supports OOP features.

Beginning with C++:**Q)What is C++? Applications of C++, a simple C++ program, C++ statements, structure of C++ program, creating, compiling and lining of file.**

1.C++ is a general-purpose programming language developed by Bjarne Stroustrup in 1983 as an extension of the C language. It supports multiple programming paradigms, including procedural, object-oriented, and generic programming. It is widely used for system-level programming, game development, and applications requiring high-performance computing.

2.Key Features: Object-Oriented Programming (OOP), Low-Level Programming, Standard Template Library (STL), Portability, Performance.

3.Applications of C++:

- i.Operating systems like Windows and parts of Linux are written in C++.
- ii. High-performance games and game engines like Unreal Engine are developed in C++.
- iii. Used to control hardware in IoT devices, cars, and medical instruments.
- iv. Core engines of browsers like Chrome and Firefox are written in C++.
- v. Simulations, real-time systems, and high-performance computing use C++.

4.Simple Code:

```
#include <iostream>

void main(){

    clrscr();

    cout<<"Hello World!";

}
```

5.C++ statements: C++ statements are commands for the program to execute.

i.Declaration Statements:

```
int x; // Declares an integer variable
```

ii.Assignment Statements:

```
x = 10; // Assigns the value 10 to x
```

iii.Input/Output Statements:

```
cin >> x; // Reads input from the user
```

```
cout << x; // Outputs the value of x
```

iv.Control Statements:

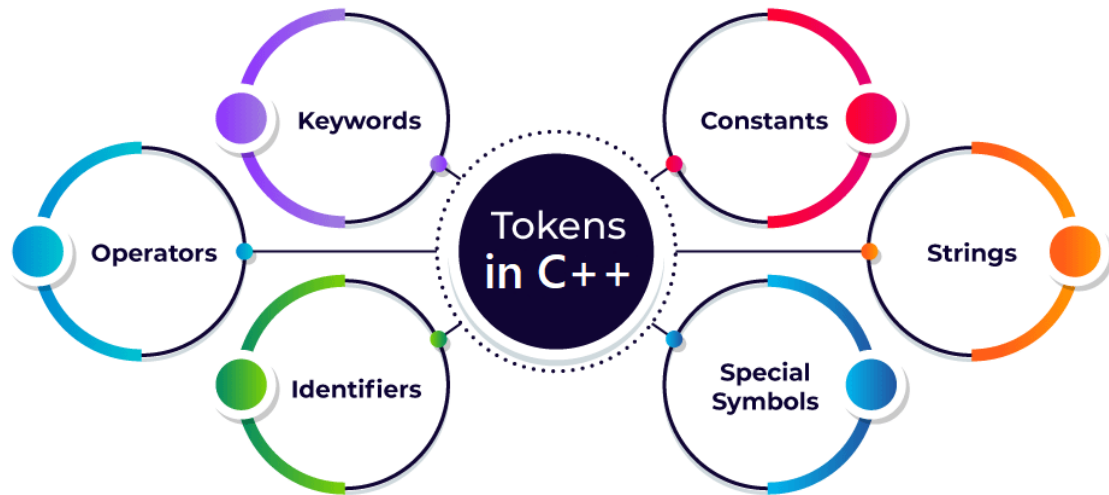
```
if (x > 5) {

    cout << "x is greater than 5";

}
```

Tokens, expressions and control structure:

Tokens:In C++, tokens can be defined as the smallest building block of C++ programs that the compiler understands. Every word in a C++ source code can be considered a token.

Types of Tokens in C++:

1. Identifiers:In C++, entities like variables, functions, classes, or structs must be given unique names within the program so that they can be uniquely identified. The unique names given to these entities are known as identifiers. Ex: name="Raju"; where name is the identifier.

2. Keywords:Keywords in C++ are the tokens that are the reserved words in programming languages that have their specific meaning and functionalities within a program. Keywords cannot be used as an identifier to name any variables.

There are 95 keywords reserved in C++. Some of the main Keywords are:

break	try	catch	char	class	const	continue
default	delete	auto	else	friend	for	float
long	new	operator	private	protected	public	return
short	sizeof	static	this	typedef	enum	throw
mutable	struct	case	register	switch	and	or
namespace	static_cast	goto	not	xor	bool	do

double	int	unsigned	void	virtual	union	while
--------	-----	----------	------	---------	-------	-------

3.Constants:Constants are the tokens in C++ that are used to define variables at the time of initialization and the assigned value cannot be changed after that. We can define the constants in C++ in two ways that are using the 'const' keyword and '#define' preprocessor directive.

i.Define Constants using the 'const' Keyword in C++:

Syntax:

```
const data_type variable_name = value;
```

ii.Define Constants using the '#define' preprocessor directive in C++:

Syntax:

```
#define constant_Name value
```

4. Strings:In C++, a **string** is not a built-in data type like 'int', 'char', or 'float'. It is a class available in the STL library which provides the functionality to work with a sequence of characters, that represents a string of text.

5.Special Symbols:Special symbols are the token characters having specific meanings within the syntax of the programming language. These symbols are used in a variety of functions, including ending the statements, defining control statements, separating items, and more.

Below are the most common special symbols used in C++ programming:

i.Semicolon (;): It is used to terminate the statement.

ii.Square brackets []: They are used to store array elements.

iii.Curly Braces {}: They are used to define blocks of code.

iv.Scope resolution (::): Scope resolution operator is used to access members of namespaces, classes, etc.

v.Dot (.): Dot operator also called member access operator used to access class and struct members.

vi.Assignment operator '=': This operator is used to assign values to variables.

vii.Double-quote ("): It is used to enclose string literals.

viii.Single-quote ('): It is used to enclose character literals.

6.Operators:C++ operators are special symbols that are used to perform operations on operands such as variables, constants, or expressions. A wide range of operators is available in C++ to perform a specific type of operations which includes arithmetic operations, comparison operations, logical operations, and more.

Operators in C

	Operators	Type
Unary Operator →	++, --	Unary Operator
	+, -, *, /, %	Arithmetic Operator
	<, <=, >, >=, ==, !=	Relational Operator
Binary Operator {	&&, , !	Logical Operator
	&, , <<, >>, ~, ^	Bitwise Operator
	=, +=, -=, *=, /=, %=	Assignment Operator
Ternary Operator →	?:	Ternary or Conditional Operator

User-defined: The data types that are defined by the user are called the derived datatype or user-defined derived data type. These types include: 1.Class, 2.Structure, 3.Union, 4.Typedef, 5.Enumeration.

Derived data types: The data types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types. These can be of four types namely: 1.Function, 2.Array, 3.Reference, 4.Pointer.

Type Compatibility: C++ is very strict with regard to type compatibility as compared to C. Type compatibility is very close to automatic or implicit type conversion. The type compatibility is being able to use two types together without modification and being able to substitute one for the other without modification.

1. Assignment compatibility
2. Expression compatibility
3. Parameter compatibility

Variable declaration: Declaration of a variable in a computer programming language is a statement used to specify the variable name and its data type. Declaration tells the compiler about the existence of an entity in the program and its location. When you declare a variable, you should also initialize it.

Dynamic initialization: In this method, the variable is assigned a value at the run-time. The value is either assigned by the function in the program or by the user at the time of running the program. The value of these variables can be altered every time the program runs. Dynamic Initialization is also known as Implicit Initialization.

Reference variables: A reference variable is one that refers to the address of another variable. It represents the name of another variable, location, or value. Once you initialize the variable references, that variable will be referred to using the variable name or reference name.

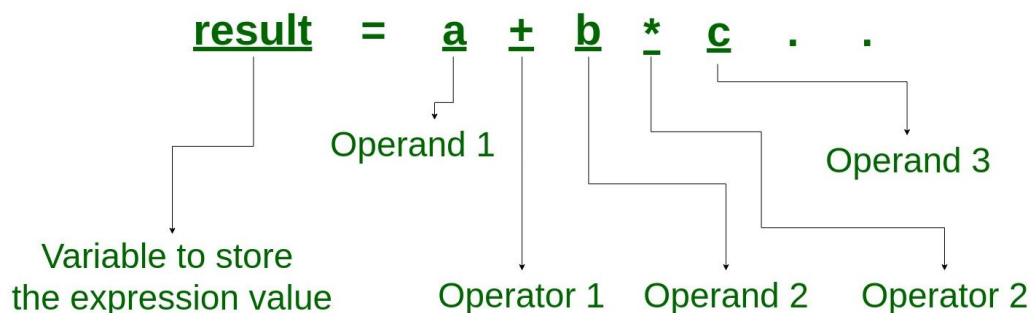
Manipulators: Manipulators are helping functions that can modify the [input/output](#) stream. It does not mean that we change the value of a variable, it only modifies the I/O stream using insertion (<<) and extraction (>>) operators.

1.Manipulators are special functions that can be included in the I/O statement to alter the format parameters of a stream.

2.Manipulators are operators that are used to format the data display.

Expressions: An expression is a combination of operators, constants and variables. An expression may consist of one or more operands, and zero or more operators to produce a value.

What is an Expression?



Implicit Conversion: Implicit Type Conversion Also known as 'automatic type conversion'.

1.Done by the compiler on its own, without any external trigger from the user.

2.Generally takes place when in an expression more than one data type is present. In such condition type conversion (type promotion) takes place to avoid lose of data.

3.All the data types of the variables are upgraded to the data type of the variable with largest data type.

bool -> char -> short int -> int ->

unsigned int -> long -> unsigned ->

long long -> float -> double -> long double

4.It is possible for implicit conversions to lose information, signs can be lost (when signed is implicitly converted to unsigned), and overflow can occur (when long long is implicitly converted to float).

Operator precedence: C++, operator precedence specifies the order in which operations are performed within an expression. When an expression contains multiple operators, those with higher precedence are evaluated before those with lower precedence.

Example:

```
int x = 5 - 17 * 6;
```

As, multiplication has higher precedence than subtraction, that's why $17 * 6$ is evaluated first, resulting in $x = -97$. If we want to evaluate $5 - 17$ first, we can use parentheses:

Example:

```
int x = (5 - 17) * 6;
```

Now $5 - 17$ is evaluated first, resulting in $x = -72$.

Control structure: Control Structures are just a way to specify flow of control in programs. Any algorithm or program can be more clear and understood if they use self-contained modules called as logic or control structures. It basically analyzes and chooses in which direction a program flows based on certain parameters or conditions. There are three basic types of logic, or flow of control, known as:

- 1.Sequence logic, or sequential flow
- 2.Selection logic, or conditional flow
- 3.Iteration logic, or repetitive flow

Operators:

Scope resolution: The Scope Resolution Operator (::) in C++ enables us to access variables, functions, or classes defined in different scopes. It helps disambiguate identifiers, making code more organized and manageable. Working with classes allows us to access class members and methods, even outside the class definition.

Member dereferencing: Dereferencing is the method where we are using a pointer to access the element whose address is being stored. We use the * operator to get the value of the variable from its address.

Memory management:

1..Memory management is a process of managing computer memory, assigning the memory space to the programs to improve the overall system performance.

2.In [C language](#), we use the malloc() or calloc() functions to allocate the memory dynamically at run time, and free() function is used to deallocate the dynamically allocated memory. [C++](#) also supports these functions, but C++ also defines unary operators such as new and delete to perform the same tasks, i.e., allocating and freeing the memory.

Type Cast:

Explicit Type Conversion: This process is also called type casting and it is user-defined. Here the user can typecast the result to make it of a particular data type.

In C++, it can be done by two ways:

1.Converting by assignment: This is done by explicitly defining the required type in front of the expression in parenthesis. This can be also considered as forceful casting.

2.Conversion using Cast operator: A Cast operator is a **unary operator** which forces one data type to be converted into another data type.

C++ supports four types of casting:

i.[Static Cast](#).

ii.Dynamic Cast.

iii.[Const Cast](#).

iv.[Reinterpret Cast](#).

Array:

Single dimension array: One-dimensional arrays are like a row of boxes where you can store things where each box can hold one item, such as a number or a word. Arrays are handy because they let you store lots of related data in one place and access it quickly.

Passing single dimension arrays to a functions: In C++, a collection of elements stored in contiguous memory locations and having the same data type is called an array. Passing arrays to functions is done to perform various operations on array elements without messing up with the main code. In C++, we have the following ways to pass an array as a parameter to the function:

1. As a sized array
2. As an unsized array
3. As a pointer (pass by pointer)
4. As a reference (pass by reference)

Null terminated strings: In C++, a null terminated character array, referred to as a C-style string, is a sequence of characters stored in contiguous memory locations where the end of the string is marked by a null character (`'\0'`).

2D Arrays: A two-dimensional array or 2D array is the simplest form of the multidimensional array. We can visualize a two-dimensional array as one-dimensional arrays stacked vertically forming a table with 'm' rows and 'n' columns. In C, arrays are 0-indexed, so the row number ranges from 0 to (m-1) and the column number ranges from 0 to (n-1).

Multidimensional Arrays: A multi-dimensional array is an array of arrays. To declare a multi-dimensional array, define the variable type, specify the name of the array followed by square brackets which specify how many elements the main array has, followed by another set of square brackets which indicates how many elements the sub-arrays have. Each set of square brackets in an array declaration adds another **dimension** to an array.

Creating and manipulating string object: C++ allows for both C-style strings (declared as character arrays) and instances of the C++ string class. Depending on the type of string, there are several methods for manipulating strings. We can use strlen to find length (length for C++ strings), strcpy (C), and copy (C++) to copy strings.